CORPORACIÓN INTERNACIONAL PARA EL DESARROLLO EDUCATIVO – CIDE

Ingeniería de Sistemas

GearEye

Documento del proceso de desarrollo del prototipo

Metodología RUP (Proceso Unificado Racional)

Leonel Serrano · Jonnatan Cubillos

Bogotá D.C., Colombia · Corte: 27 de octubre de 2026 · Entrega: 17 de noviembre de 2026

Versión 1.0 — prototipo

1. Propósito del documento

Este documento explica cómo se construyó el prototipo de GearEye. Sigue las cuatro fases de RUP que usa el cronograma del proyecto: Inicio, Elaboración, Construcción y Transición. Para cada fase indica qué se hizo, qué artefacto lo respalda y en qué fecha quedó terminado.

La fuente de verdad es el código del repositorio y su historial de Git. Cuando la propuesta original (documento «GearEye Proyecto») describe algo distinto de lo que se implementó, la sección 8 lo aclara.

2. Metodología RUP aplicada

RUP organiza el trabajo en fases que terminan en un hito. Dentro de cada fase se trabaja por iteraciones cortas: cada iteración produce una parte del sistema que funciona y se prueba. En GearEye cada iteración corresponde a uno o varios commits en Git.

Fase	Pregunta que responde	Hito de cierre	Fechas
Inicio	¿Qué problema resolvemos y hasta dónde llega el prototipo?	Problema, objetivos y alcance aprobados	07/09/2026 – 13/09/2026
Elaboración	¿Cómo será el sistema?	Requerimientos, casos de uso, arquitectura y base de datos definidos	14/09/2026 – 12/10/2026
Construcción	¿Funciona cada módulo y funcionan juntos?	Versión integrada del prototipo	12/10/2026 – 16/11/2026
Transición	¿Está listo para entregarse?	Prototipo probado y entregado	16/11/2026 – 17/11/2026

Las disciplinas de RUP se cubrieron así: modelado del negocio y requerimientos (documento del proyecto y secciones 3–4 de este documento), análisis y diseño (arquitectura y modelo entidad-relación), implementación (código en Backend/ y Frontend/), pruebas (carpeta tests/ y plan de la sección 6), despliegue (guías de Proxmox) y gestión de configuración (Git).

3. Fase de Inicio

3.1 Problema

Las PYMES e instituciones educativas no suelen tener una herramienta que muestre qué equipos están conectados a su red, cuáles dejaron de responder y por qué. Las herramientas comerciales son costosas o complejas. El diagnóstico se hace a mano y tarde: el equipo técnico se entera de una falla cuando un usuario la reporta.

3.2 Objetivos

Objetivo general: desarrollar un sistema de monitoreo de red que detecte los dispositivos conectados, muestre su estado y la topología, y apoye el diagnóstico de fallas con análisis automático.

Objetivos específicos:

1. Descubrir automáticamente los dispositivos de una red LAN con su IP, MAC, nombre y fabricante.
2. Medir de forma periódica la disponibilidad y la latencia de cada dispositivo y guardar el historial.
3. Representar la topología lógica de la red y, cuando el switch lo permita, los enlaces físicos por LLDP.
4. Generar alertas cuando aparece un equipo nuevo, cuando uno desaparece o cuando su latencia supera un umbral.
5. Diagnosticar la causa probable de las fallas comparando cada equipo con su comportamiento normal aprendido.
6. Ofrecer un panel web protegido con autenticación de dos factores y roles de usuario.

3.3 Alcance del prototipo

Incluido en el prototipo	Fuera del alcance
Descubrimiento por tabla ARP y ping (ICMP)	Cambiar configuraciones de red o bloquear equipos
Historial de latencia y disponibilidad (180 días)	IDS completo o análisis del contenido del tráfico
Alertas en el panel y bitácora exportable en CSV	Alertas por correo o mensajería (solo se envían invitaciones)
Topología lógica y física (LLDP por SNMP)	Tráfico por interfaz (bytes TX/RX) por SNMP
Diagnóstico IA con línea base y reglas	Agente para Linux o macOS (el agente actual usa PowerShell)
Varias empresas, agente por red y despliegue en Proxmox	Firewall perimetral o WAF propios

4. Fase de Elaboración

4.1 Actores

Actor	Descripción	Qué necesita
Administrador general	Administra la plataforma y crea empresas.	Crear empresas, invitar administradores, ver el estado de los agentes.
Administrador de empresa	Responsable técnico de la red de una empresa.	Ver la red, configurar el monitoreo, reconocer alertas, marcar equipos confiables, ejecutar diagnósticos.
Usuario de consulta	Persona que solo observa.	Ver inventario, alertas, rendimiento y diagnóstico IA sin modificar nada.
Agente de red	Programa instalado dentro de la LAN de cada empresa.	Escanear la red y enviar resultados al servidor central con su token.

4.2 Requerimientos

Requerimientos funcionales:

Código	Requerimiento	Dónde se cumple
RF-01	Descubrir los dispositivos de la subred con IP, MAC, nombre, fabricante y tipo.	Backend/Services/Red.py
RF-02	Clasificar cada dispositivo como activo, detectado o inactivo y medir su latencia.	Red.py · Models/Dispositivo.py
RF-03	Guardar cada escaneo y conservar el historial 180 días.	Services/Historial.py
RF-04	Registrar eventos: equipo nuevo, cambio de estado, equipo ausente, latencia elevada.	Historial.registrar_eventos_escaneo
RF-05	Mostrar alertas pendientes y permitir reconocerlas.	/api/alertas · panel Alertas
RF-06	Mantener un inventario de equipos confiables.	/api/dispositivos/{ip}/confianza
RF-07	Construir el mapa topológico lógico y físico (LLDP).	Services/Topologia.py · panel Topología
RF-08	Diagnosticar la causa probable de fallas y anomalías.	Services/AnalisisIA.py · /api/analisis-ia
RF-09	Ejecutar un diagnóstico avanzado bajo demanda (ping ×5, puertos, ruta).	Services/Diagnostico.py
RF-10	Exportar inventario, informe completo y bitácora en CSV.	/api/informes.csv · /api/eventos.csv
RF-11	Programar el monitoreo cada 1, 5, 15 o 30 minutos y configurar umbrales.	/api/configuracion/monitoreo · agent.py
RF-12	Administrar empresas, usuarios por invitación y agentes.	Authentication.py · admin.html

Requerimientos no funcionales:

Código	Atributo	Requerimiento
RNF-01	Seguridad	Acceso con contraseña y código TOTP; la sesión vence tras 5 minutos sin actividad.
RNF-02	Seguridad	Contraseñas guardadas como hash; los agentes se autentican con token propio.
RNF-03	Privacidad	Solo se recogen datos técnicos (IP, MAC, latencia); no se lee el contenido del tráfico.
RNF-04	Aislamiento	Los datos de cada empresa quedan separados (esquema PostgreSQL o archivo SQLite propio).
RNF-05	Rendimiento	Un escaneo de una subred /24 se completa en segundos usando hilos en paralelo.
RNF-06	Disponibilidad	Si la base de datos falla, la API responde 503 con un mensaje claro.
RNF-07	Usabilidad	Panel en español, adaptable a móvil, con manual de usuario integrado.
RNF-08	Portabilidad	Servidor en contenedor Docker; base de datos PostgreSQL o SQLite local.

4.3 Casos de uso

Código	Caso de uso	Actor	Flujo principal
CU-01	Iniciar sesión	Todos	Ingresa usuario y contraseña → ingresa código del autenticador → accede al panel.
CU-02	Escanear la red	Admin. empresa / Agente	Solicita escaneo → el sistema lee ARP y hace ping → guarda resultados → muestra inventario.
CU-03	Consultar inventario	Todos	Abre Dispositivos → filtra por estado, tipo o latencia → abre el detalle de un equipo.
CU-04	Revisar alertas	Admin. empresa	Abre Alertas → lee la causa → reconoce la alerta.
CU-05	Marcar equipo confiable	Admin. empresa	Abre el detalle → «Marcar como confiable» → el equipo deja de contarse como desconocido.
CU-06	Ver diagnóstico IA	Todos	Abre Diagnóstico IA → ve hallazgos ordenados por severidad, causa probable y recomendación.
CU-07	Ver topología	Todos	Abre Topología → ve el gateway y los nodos; si hay LLDP, los puertos físicos.
CU-08	Exportar informe	Todos	Elige periodo → descarga CSV con inventario, mediciones, eventos y resumen.
CU-09	Administrar empresas	Admin. general	Crea empresa → invita administrador → entrega token del agente.

4.4 Arquitectura

GearEye usa una arquitectura cliente-servidor con un agente por red:

- Agente de red (Backend/agent.py): se ejecuta en un equipo Windows dentro de la LAN. Escanea con ARP e ICMP, consulta LLDP por SNMP y envía el resultado por HTTPS al servidor.
- API (Backend/GearEye.py, FastAPI): recibe los escaneos, guarda datos, calcula alertas y diagnóstico, y sirve el panel.
- Servicios: Red (descubrimiento), Historial (persistencia y eventos), Topologia (LLDP/SNMP), Diagnostico (pruebas por equipo) y AnalisisIA (diagnóstico automático).
- Base de datos: PostgreSQL en producción; SQLite local para desarrollo.
- Frontend (HTML, CSS y JavaScript sin framework): panel de monitoreo y panel de administración.
- Despliegue: VM Debian en Proxmox con Docker Compose y Cloudflare Tunnel para publicar HTTPS.

Flujo del sistema: Captura → Procesamiento → Diagnóstico IA → Alerta → Registro → Visualización.

4.5 Modelo de datos

Tabla	Contenido
networks	Redes (CIDR, gateway) y su configuración de monitoreo y alertas.

devices	Inventario: IP, MAC, nombre, fabricante, tipo, estado, responsable y notas.
scan_runs	Cada ejecución de escaneo.
measurements	Estado y latencia de cada equipo en cada escaneo (base del aprendizaje de la IA).
events	Bitácora de cambios y alertas, con fecha de reconocimiento.
trusted_devices	Equipos que el administrador marcó como confiables.
diagnostics	Resultados de diagnóstico por equipo.
network_topology	Último mapa LLDP recibido por red.

El diagrama completo está en Docs/Modelo_Entidad_Relacion.md y el script en Database/modelo_geareye_postgresql.sql.

5. Fase de Construcción

La construcción se hizo en iteraciones. La tabla resume cada una con su evidencia en Git.

Iteración	Fecha	Commit	Resultado
1. Base del proyecto	21/09/2026	883cfa4, eb96ac7	Estructura del repositorio y API FastAPI inicial.
2. Comprobación de red	22/09/2026	3ce70ad	Ping a una IP desde la API.
3. Descubrimiento	24/09/2026	bbf7b9a, 32c2ccd	Escaneo de la subred y modelo Dispositivo.
4. Identificación	06/10/2026	af36cca	Fabricante por MAC, tipo de equipo y diagnóstico por umbrales.
5. Monitoreo y panel	20/10/2026	8e96278, 4715f5c	Autenticación 2FA, historial, eventos, alertas, base de datos y dashboard.
6. Integración multiempresa	21/10/2026	09366c1	Agente LAN, administración de empresas y despliegue en Proxmox.
7. Diagnóstico IA	27/10/2026	pendiente de commit	Motor AnalisisIA, panel «Diagnóstico IA», pruebas automáticas y correcciones.

5.1 Módulo de descubrimiento

Lee la configuración del adaptador activo (prioriza Ethernet sobre Wi-Fi), recorre la subred con ping en paralelo y completa la información con la tabla ARP. El fabricante se obtiene del prefijo de la MAC. Un equipo es «activo» si responde al ping, «detectado» si solo aparece en ARP e «inactivo» si no hay rastro.

5.2 Módulo de monitoreo

Cada escaneo se guarda como una ejecución con sus mediciones. Al compararlo con el anterior se registran eventos: equipo nuevo, cambio de estado, equipo ausente y latencia sobre el umbral. El monitoreo puede ser manual o programado; en redes remotas lo ejecuta el agente según la configuración guardada.

5.3 Módulo de topología

Muestra el gateway como nodo raíz y los equipos como nodos hoja. Si un switch gestionable publica sus vecinos por LLDP y el agente tiene credenciales SNMP de solo lectura (v3 recomendado), el mapa muestra los enlaces físicos con sus puertos. GearEye no inventa enlaces físicos: sin LLDP muestra solo la vista lógica.

5.4 Motor de diagnóstico con IA

El motor (Backend/Services/AnalisisIA.py) aprende el comportamiento normal de cada equipo a partir de su historial y lo combina con reglas de correlación. No usa servicios externos.

Aprendizaje de la línea base:

- Con al menos 5 escaneos con respuesta, calcula la mediana de latencia del equipo y su desviación absoluta mediana (MAD).
- Una medición es anómala si su puntuación Z robusta (0,6745 × desviación / MAD) es ≥ 3,5 y supera la mediana en al menos 20 ms.
- El umbral es distinto para cada equipo y se ajusta solo a medida que llegan escaneos, lo que cubre la «automatización de umbrales» de la sección 3.4.4 de la propuesta.

Reglas de diagnóstico (sección 3.4.3 de la propuesta):

Situación observada	Causa probable	Clasificación
El gateway no responde	Falla del router o de toda la red	Crítico
3 o más equipos del mismo switch (según LLDP) caen a la vez	Falla de ese switch o de su uplink	Crítico
La mitad o más de los equipos que respondían caen a la vez	Caída de un segmento de la LAN	Crítico
Un equipo con latencia fuera de su línea base	Congestión puntual o problema de cable o señal	Advertencia
La mayoría de los equipos con latencia anómala	Congestión general o bucle de red	Advertencia
3 o más cambios de estado en los últimos 10 escaneos	Conexión intermitente	Advertencia
La MAC del gateway responde también por otra IP	Posible envenenamiento ARP	Intrusión
Una IP cambia de MAC	IP reasignada o suplantación	Advertencia (Intrusión si es el gateway)
MAC fuera del inventario confiable	Posible equipo intruso	Intrusión

Cada nodo recibe un estado de la sección 5.4 de la propuesta: activo, intermitente, caído o intruso detectado. El panel muestra los hallazgos ordenados por severidad, con la causa probable, la recomendación y la evidencia.

5.5 Backend, seguridad y dashboard

- API REST en FastAPI con rutas protegidas por sesión y por rol.
- Autenticación con contraseña con hash y código TOTP; la sesión vence a los 5 minutos de inactividad.
- Datos de cada empresa aislados; los agentes usan un token propio.
- Panel web con red local, topología, inventario, alertas, diagnóstico IA, rendimiento e informes CSV.

6. Fase de Transición

6.1 Pruebas automáticas

La carpeta tests/ tiene 15 pruebas con pytest que se ejecutan con una base SQLite temporal, sin tocar datos reales. Cubren la línea base, cada regla del motor de IA y el endpoint /api/analisis-ia con datos guardados. Resultado al 27/10/2026: 15 de 15 pruebas aprobadas.

Las pruebas encontraron un error real: al registrar eventos en SQLite se abría una segunda conexión mientras la primera tenía una transacción abierta, y la operación quedaba bloqueada («database is locked»). Se corrigió leyendo la configuración antes de abrir la conexión de escritura.

6.2 Plan de pruebas del prototipo (16 y 17 de noviembre)

Prueba	Cómo se ejecuta	Resultado esperado
Descubrimiento	Escanear una red con equipos conocidos.	Aparecen todos los equipos encendidos con IP y MAC.
Detección de fallas	Desconectar un equipo y escanear.	Evento «dispositivo ausente» y alerta pendiente.
Caída del gateway	Simular gateway sin respuesta (red de pruebas).	Hallazgo crítico «El gateway no responde».
Latencia anómala	Saturar el enlace de un equipo tras 5 escaneos normales.	Hallazgo «Latencia fuera de su comportamiento normal».
Intrusos	Marcar equipos confiables y conectar uno nuevo.	Hallazgo «Equipo no reconocido en la red».
Alertas	Reconocer una alerta desde el panel.	La alerta sale de la lista de pendientes.
Seguridad	Llamar a la API sin sesión.	Respuesta 401.
Estabilidad	Monitoreo cada minuto durante 2 horas.	Sin errores en el registro y escaneos continuos.

6.3 Entrega

La entrega del 17/11/2026 incluye: el repositorio con el código, este documento, el cronograma actualizado (Docs/Cronograma_GearEye_RUP.xlsx), el manual de usuario (/manual) y la guía de despliegue en Proxmox.

7. Cómo ejecutar el prototipo

7. Instalar Python 3.10 o superior.
8. Instalar dependencias: pip install -r requirements.txt
9. Iniciar el servidor: uvicorn Backend.GearEye:app --host 127.0.0.1 --port 8000
10. Abrir http://127.0.0.1:8000/app/ y crear la cuenta con el autenticador.
11. Ejecutar las pruebas: pip install -r requirements-dev.txt y luego python -m pytest tests

8. Diferencias entre la propuesta y el prototipo

Estas diferencias deben reflejarse en el documento del proyecto para que describa lo que realmente se construyó.

Propuesta original	Prototipo construido	Motivo
Isolation Forest con scikit-learn (cap. 6.4)	Línea base estadística por equipo (mediana y MAD) más reglas de correlación	Funciona con pocos datos, explica cada hallazgo y no añade dependencias pesadas. Isolation Forest queda como mejora futura.
MySQL o MongoDB	PostgreSQL (producción) y SQLite (local)	PostgreSQL permite separar empresas por esquema; SQLite facilita el desarrollo.
Flask o FastAPI	FastAPI	Validación automática de datos y documentación Swagger.
React o Vue.js	HTML, CSS y JavaScript sin framework	Panel ligero sin proceso de compilación.
Alertas por correo	Alertas en el panel y bitácora CSV; el correo solo envía invitaciones	Pendiente para una versión futura.
Tráfico TX/RX por SNMP	SNMP solo para vecinos LLDP	Se priorizó la topología física.
SSH y MQTT	No implementados	Fuera del alcance del prototipo.
Firewall perimetral, lógico y WAF propios	HTTPS mediante Cloudflare Tunnel y sesiones protegidas	Un firewall propio no es parte de un sistema de monitoreo.
Agente multiplataforma	Agente para Windows (PowerShell)	El descubrimiento usa comandos de Windows.

9. Estado actual y próximos pasos

Al 27/10/2026 el prototipo tiene un avance estimado del 81 %. Las fases de Inicio y Elaboración están completas. En Construcción, el descubrimiento, el monitoreo y el dashboard están terminados; el diagnóstico IA está implementado y probado con datos simulados, y la integración está en curso.

12. Validar el motor de IA con datos de una red real durante varios días (antes del 09/11).
13. Probar el flujo completo con el agente instalado en una red de pruebas (09/11 – 16/11).
14. Ejecutar el plan de pruebas de la sección 6.2 y registrar los resultados (16/11 – 17/11).

15. Actualizar el documento del proyecto con las diferencias de la sección 8.